AI Assignment 2

Faiza Salman Khatri fs09192

Ali Athar ma10569

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
# Data Generation
c = 1e-3

def target_function(x, y, c=c):
    # f(x, y) = exp(-(x^2 + y^2) / 8) * cos(sqrt(x^2 + y^2) + c)
    r2 = x**2 + y**2
    return np.exp(-r2/8) * np.cos(np.sqrt(r2) + c)


def generate_data(start, end, step=0.2):

    v = np.round(np.arange(start, end+step/2, step), 10)
    xx, yy = np.meshgrid(v,v)
    X = np.column_stack([xx.ravel(), yy.ravel()])
    y = target_function(X[:,0], X[:,1]).reshape(-1,1)
    return X,y,v


X_train, y_train, grid_train = generate_data(-2, 2)
X_test, y_test, grid_test = generate_data(-1.9, 1.9)

print("Train:", X_train.shape, y_train.shape, "| grid:", grid_train[:3], "...",
grid_train[-1])
print("Test: ", X_test.shape, y_test.shape, "| grid:", grid_test[:3], "...",
grid_test[-1])
```

```
Train: (441, 2) (441, 1) | grid: [-2.  -1.8 -1.6] ... 2.0
Test:  (400, 2) (400, 1) | grid: [-1.9 -1.7 -1.5] ... 1.9
```

Data Generation:

x and y both go from -2 to 2 in steps of 0.2, which gives 21 values each. np.meshgrid pairs every x with every y, so the network sees the whole square and not just one line through it. This gives 441 training points, stored as a 441 x 2 input matrix, with f(x, y) as the target.

The test set uses the same step from -1.9 to 1.9, which gives 400 points. These sit exactly between the training points, so the network is never tested on a point it trained on.

np.arange with a step of 0.2 gives values like 0.20000000000000018 because of floating point error, so the values are rounded to 10 decimal places.

In [3]:
```
ACTIVATIONS = {
    "tansig": (np.tanh,                          lambda a: 1 - a**2),
    "logsig": (lambda z: 1 / (1 + np.exp(-z)),   lambda a: a * (1 - a)),
    "relu":   (lambda z: np.maximum(0, z),       lambda a: (a >
0).astype(float)),
}

def forward(model, X):
```

```
    g = ACTIVATIONS[model["activation"]][0]
    A = [X]
    n_layers = len(model["W"])
    for l in range(n_layers):
        z = A[-1] @ model["W"][l] + model["b"][l]
        A.append(z if l == n_layers - 1 else g(z))   # linear output layer
    return A

# Model Training
def train(X_train, y_train,
                epochs=5000,
                lr=0.05,
                hidden_layers=[40],
                activation="tansig",
                bias_init=0.5,
                seed = 0):
    """
    Parameters:
        X_train, y_train : training data
        epochs : number of iterations
        lr : learning rate
        hidden_layers : list of hidden layer sizes, e.g. [40] or [64,32]
        activation : activation function name ('tansig', 'logsig', 'relu', etc.)
        bias_init : initial bias value (float)

    Returns:
        model_params, losses
    """
    rng = np.random.default_rng(seed)
    sizes = [X_train.shape[1]] + list(hidden_layers) + [y_train.shape[1]]

    # initialize weights (Xavier uniform) and biases (constant bias_init)
    W = [rng.uniform(-1, 1, (n_in, n_out)) * np.sqrt(6 / (n_in + n_out))
        for n_in, n_out in zip(sizes[:-1], sizes[1:])]
    b = [np.full((1, n_out), float(bias_init)) for n_out in sizes[1:]]
    model = {"W": W, "b": b, "activation": activation}

    # select activation derivative
    g_prime = ACTIVATIONS[activation][1]

    losses = []
    N = len(X_train)
    for epoch in range(epochs):
        # forward pass
        A = forward(model, X_train)
        y_hat = A[-1]

        loss = np.mean((y_hat - y_train) ** 2)
        if not np.isfinite(loss):
            print(f"  diverged at epoch {epoch} (lr={lr}, activation=
{activation})")
            break
        losses.append(loss)

        # backward pass
```

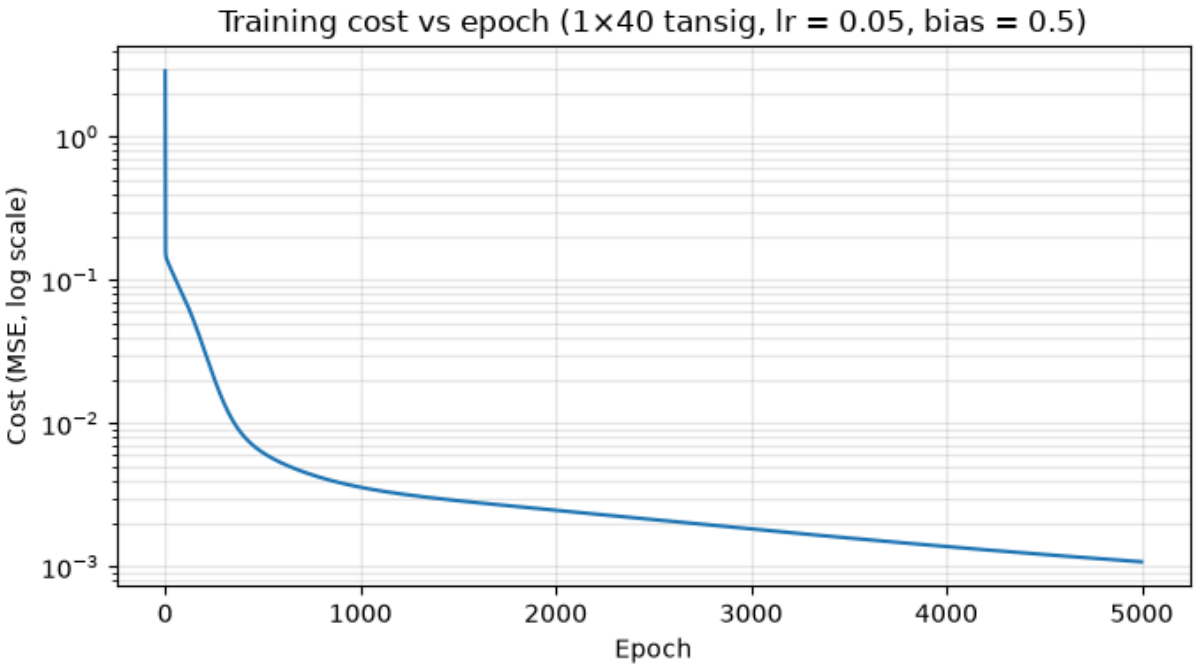
```
        delta = 2 * (y_hat - y_train) / N
        for l in range(len(W) - 1, -1, -1):
            grad_W = A[l].T @ delta
            grad_b = delta.sum(axis=0, keepdims=True)
            if l > 0:
                delta = (delta @ W[l].T) * g_prime(A[l])   # uses W[l] before it
is updated
            W[l] -= lr * grad_W
            b[l] -= lr * grad_b

    return model, np.array(losses)

# call train and capture losses
model, losses = train(X_train, y_train, epochs=5000, lr=0.05,
                      hidden_layers=[40], activation="tansig", bias_init=0.5)
print(f"Final training MSE: {losses[-1]:.3e}")

# plot cost vs epoch
plt.figure(figsize=(7, 4))
plt.semilogy(losses)
plt.xlabel("Epoch")
plt.ylabel("Cost (MSE, log scale)")
plt.title("Training cost vs epoch (1×40 tansig, lr = 0.05, bias = 0.5)")
plt.grid(True, which="both", alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Final training MSE: 1.077e-03
```



Model Training:

Network: 2 inputs, one hidden layer of 40 tansig neurons, and 1 linear output neuron. The output is linear because this is regression, so the output needs to be able to take any value.

Settings: learning rate 0.05, 5000 epochs, and every bias starts at 0.5. The weights start small and random using Xavier initialisation, which scales them by layer size so the signal does not blow up or die out as it goes through the network.

Cost: mean squared error. Each epoch runs all 441 points forward, backpropagates the error layer by layer to get the gradient for every weight and bias, and takes one gradient descent step. hidden_layers is a list, so the same code handles [40], [40, 40] and so on in part d.

What the cost curve shows:

The cost drops from about 3 to about 0.15 in the first few epochs, as the network fixes its overall offset. It then falls steeply until about epoch 500 while it learns the shape of the function. After that it keeps falling slowly and ends at a training MSE of 1.08e-3. It never goes back up, so the learning rate is a sensible size. It is still falling at epoch 5000, so more epochs would help a little.

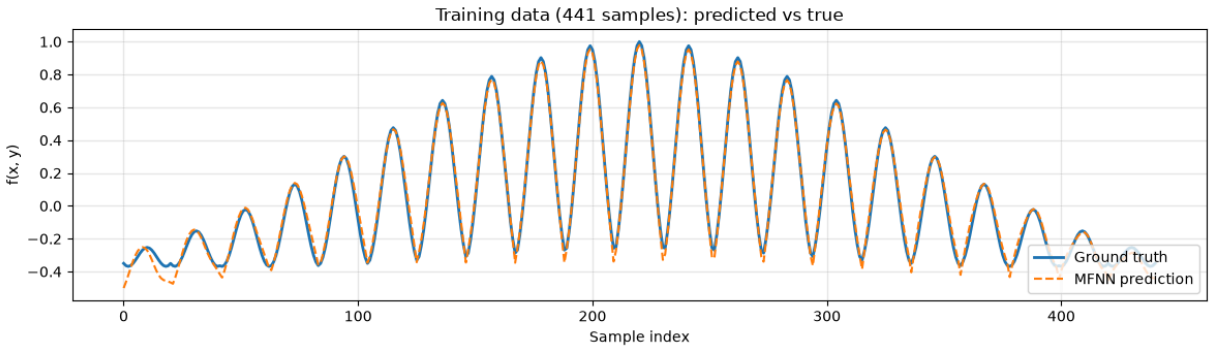
In [4]:
```
# Model Testing
def predict(model, X_test):
    # forward pass only
    return forward(model, X_test)[-1]

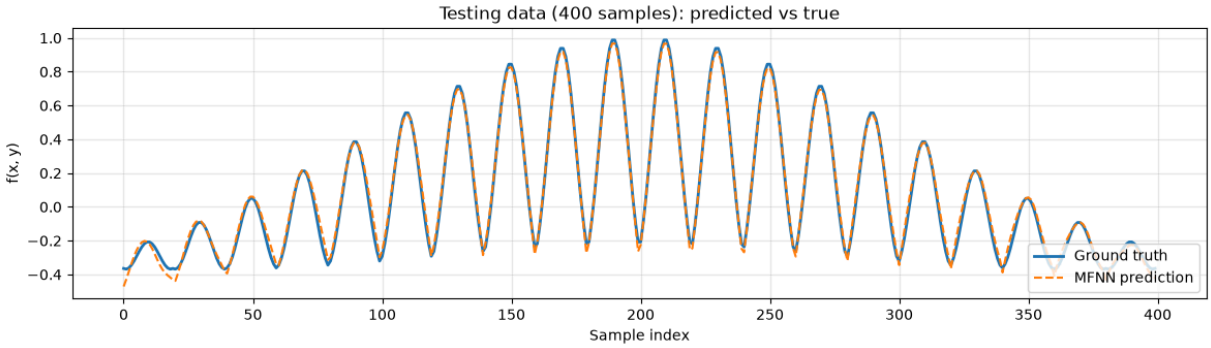
def mse(a, b):
    return np.mean((a - b) ** 2)

# run predictions on train & test
pred_train = predict(model, X_train)
pred_test = predict(model, X_test)
print(f"Train MSE: {mse(pred_train, y_train):.3e}   max |error|:
{np.max(np.abs(pred_train - y_train)):.3f}")
print(f"Test  MSE: {mse(pred_test, y_test):.3e}   max |error|:
{np.max(np.abs(pred_test - y_test)):.3f}")

# plot predicted vs ground truth: every sample, train and test kept separate
fig, axes = plt.subplots(2, 1, figsize=(12, 7))
for ax, y_true, y_pred, name in [(axes[0], y_train, pred_train, "Training"),
(axes[1], y_test, pred_test, "Testing")]:
    ax.plot(y_true, label="Ground truth", linewidth=2)
    ax.plot(y_pred, "--", label="MFNN prediction", linewidth=1.5)
    ax.set_title(f"{name} data ({len(y_true)} samples): predicted vs true")
    ax.set_xlabel("Sample index")
    ax.set_ylabel("f(x, y)")
    ax.legend(loc="lower right")
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Train MSE: 1.077e-03   max |error|: 0.150
Test  MSE: 7.437e-04   max |error|: 0.107
```





Model Testing:

Train: MSE 1.08e-3, largest single error 0.150.

Test: MSE 7.44e-4, largest single error 0.107.

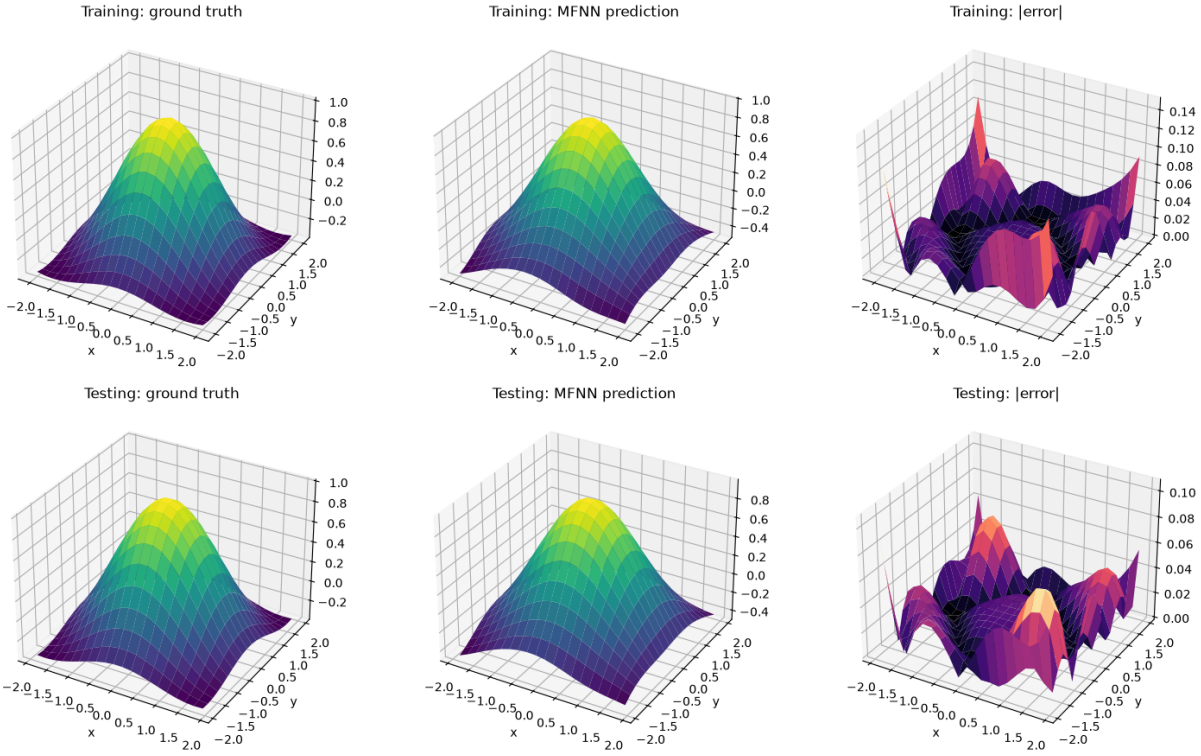
f ranges from about -0.37 to 1, and an MSE of 1e-3 means a typical prediction is off by about 0.03. The train and test errors are close, so the network learned the function and did not just memorise the training points.

The plot shows every sample in order. The wave pattern comes from how the grid was flattened: each wave is one sweep across x for a fixed y, and the waves are tallest in the middle because the peak of f is at the origin. The prediction follows the truth almost everywhere. The visible misses are at the very start (a corner of the square) and at some of the troughs, where the network dips slightly too low.

In [5]:
```
def as_grid(values, grid):
    return values.reshape(len(grid), len(grid))

fig = plt.figure(figsize=(15, 9))
panels = [
    (grid_train, y_train, "Training: ground truth"),
    (grid_train, pred_train, "Training: MFNN prediction"),
    (grid_train, np.abs(pred_train - y_train), "Training: |error|"),
    (grid_test, y_test, "Testing: ground truth"),
    (grid_test, pred_test, "Testing: MFNN prediction"),
    (grid_test, np.abs(pred_test - y_test), "Testing: |error|"),
]
for i, (grid, values, title) in enumerate(panels):
    ax = fig.add_subplot(2, 3, i + 1, projection="3d")
    xx, yy = np.meshgrid(grid, grid)
```

```
    ax.plot_surface(xx, yy, as_grid(values, grid), cmap="viridis" if "error" not
in title else "magma")
    ax.set_title(title)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
plt.tight_layout()
plt.show()
```



The predicted surface is almost the same as the true one: the same central bump and the same fall off towards the edges. The error plots show that the network struggles most at the corners. That is where the function curves back up and where there are the fewest neighbouring training points to pin the fit down.

This is also why the test error is lower than the training error. The test grid stops at ±1.9, so it never reaches the corners where the largest errors are.

In [6]:
```
# Exploring hyperparameters
# Suggested learning rates to compare: 0.01, 0.1, 0.3
# Also compare: number of hidden layers, activation function (tansig vs logsig vs
relu), initial bias value

import pandas as pd

BASELINE = dict(epochs=5000, lr=0.05, hidden_layers=[40], activation="tansig",
bias_init=0.5)

def run_experiment(param, values, label=str):
    rows, curves = [], {}
    for v in values:
        cfg = {**BASELINE, param: v}
        with np.errstate(over="ignore", invalid="ignore"):
```

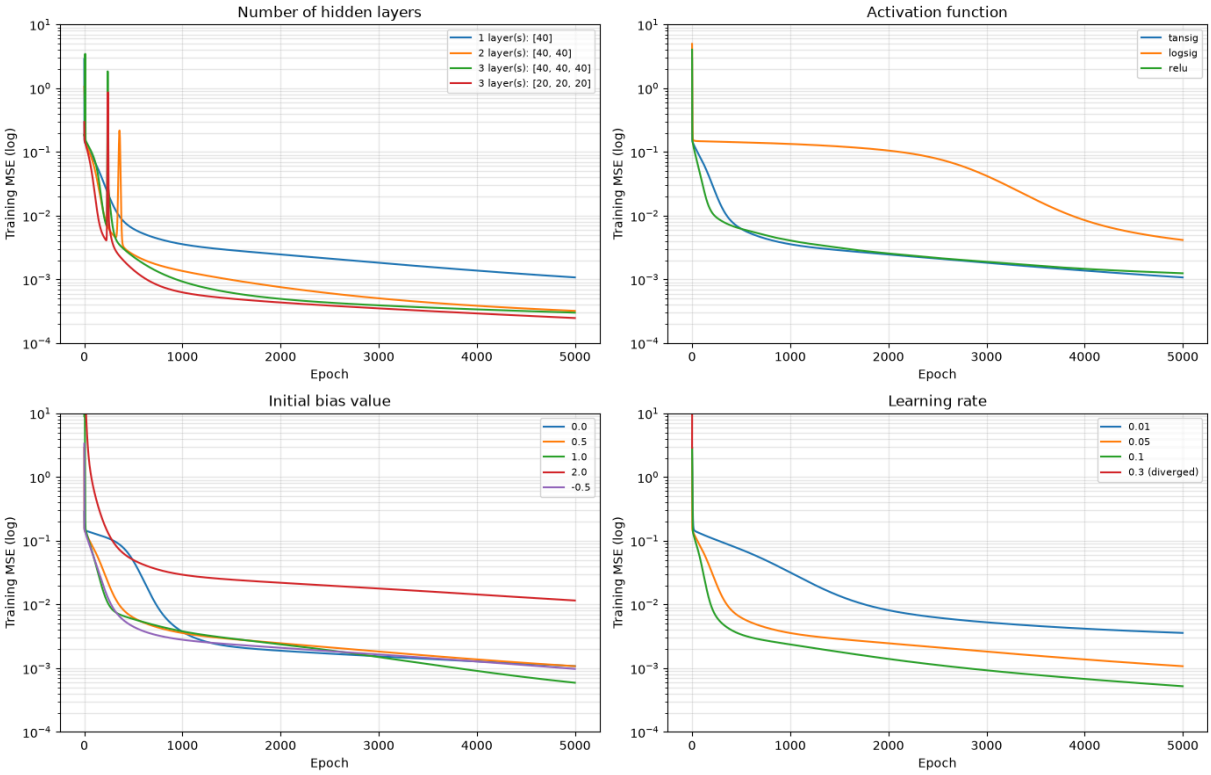
```
            m, l = train(X_train, y_train, **cfg)
        diverged = len(l) < cfg["epochs"]
        rows.append({param: label(v),
                     "train MSE": np.nan if diverged else mse(predict(m, X_train),
y_train),
                     "test MSE": np.nan if diverged else mse(predict(m, X_test),
y_test),
                     "diverged": diverged})
        curves[label(v)] = l
    return pd.DataFrame(rows), curves

experiments = {
    "Number of hidden layers": ("hidden_layers", [[40], [40, 40], [40, 40, 40],
[20, 20, 20]],
                                lambda v: f"{len(v)} layer(s): {v}"),
    "Activation function":     ("activation", ["tansig", "logsig", "relu"], str),
    "Initial bias value":      ("bias_init", [0.0, 0.5, 1.0, 2.0, -0.5], str),
    "Learning rate":           ("lr", [0.01, 0.05, 0.1, 0.3], str),
}

all_results = {}
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
for ax, (title, (param, values, label)) in zip(axes.ravel(), experiments.items()):
    table, curves = run_experiment(param, values, label)
    all_results[title] = table
    for name, l in curves.items():
        ax.semilogy(np.minimum(l, 1e3), label=name + (" (diverged)" if len(l) <
BASELINE["epochs"] else ""))
    ax.set_title(title)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Training MSE (log)")
    ax.set_ylim(1e-4, 10)   # fixed range so a diverging run doesn't blow up the
axis
    ax.grid(True, which="both", alpha=0.3)
    ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

for title, table in all_results.items():
    print(f"\n=== {title} ===")
    print(table.to_string(index=False, float_format=lambda x: f"{x:.2e}"))
```

```
diverged at epoch 128 (lr=0.3, activation=tansig)
```




```
=== Number of hidden layers ===
           hidden_layers  train MSE  test MSE  diverged
       1 layer(s): [40]   1.08e-03  7.44e-04     False
    2 layer(s): [40, 40]   3.21e-04  2.32e-04     False
3 layer(s): [40, 40, 40]   3.01e-04  2.10e-04     False
3 layer(s): [20, 20, 20]   2.48e-04  1.58e-04     False

=== Activation function ===
activation  train MSE  test MSE  diverged
    tansig   1.08e-03  7.44e-04     False
    logsig   4.16e-03  3.43e-03     False
      relu   1.25e-03  9.26e-04     False

=== Initial bias value ===
bias_init  train MSE  test MSE  diverged
      0.0   1.08e-03  7.02e-04     False
      0.5   1.08e-03  7.44e-04     False
      1.0   5.93e-04  4.17e-04     False
      2.0   1.16e-02  1.20e-02     False
     -0.5   9.77e-04  6.76e-04     False

=== Learning rate ===
  lr  train MSE  test MSE  diverged
0.01   3.59e-03  2.84e-03     False
0.05   1.08e-03  7.44e-04     False
 0.1   5.22e-04  3.40e-04     False
 0.3        NaN       NaN      True
```

Fine-Tuning:

Each experiment changes one setting and keeps the rest at the baseline (1 x 40, tansig, lr 0.05, bias 0.5, 5000 epochs). All runs use the same random seed, so differences smaller than

about 1.5x could just be chance.

Number of layers:

1 layer of 40: test MSE 7.44e-4 (161 weights and biases)

2 layers of 40: 2.32e-4 (1,801)

3 layers of 40: 2.10e-4 (3,441)

3 layers of 20: 1.58e-4 (921)

Depth made the biggest difference. A second layer cut the test error by about 3x, because each layer can combine the features learned by the one before it. The gains shrink quickly, and three layers of 40 is barely better than two. Three layers of 20 did best while using about half the parameters of two layers of 40, so depth matters more than width here. The deeper networks were less stable early on, with sharp loss spikes in the first few hundred epochs, and they are slower per epoch.

Activation function:

tansig: 7.44e-4

relu: 9.26e-4

logsig: 3.43e-3

tansig is best and relu is close. relu builds its output from straight line pieces, so it fits the smooth peak slightly worse. logsig is about 4 to 5x worse. Its loss sat on a plateau around 0.15 for about 2,500 epochs before it started improving. The sigmoid's slope is at most 0.25 (tanh's is 1), so the gradients are smaller and learning is slower. Its output is also always positive instead of centred on zero, which makes the weight updates less efficient.

Biases:

-0.5: 6.76e-4

0.0: 7.02e-4

0.5: 7.44e-4

1.0: 4.17e-4

2.0: 1.20e-2

Between -0.5 and 1.0 the starting bias barely matters, because backpropagation quickly moves the biases to where they need to be. The differences between those four are small enough to be chance. A bias of 2.0 is about 16x worse. Every hidden neuron starts with a large input, which pushes tanh into its flat region where the slope is close to zero (tanh'(2) ≈

0.07). The gradients are tiny, so those neurons barely learn. This is called saturation, and it is the same reason logsig was slow.

Learning rate:

0.01: 2.84e-3

0.05: 7.44e-4

0.1: 3.40e-4

0.3: diverged at epoch 128

0.01 takes steps that are too small, and its curve is still falling at epoch 5000. 0.1 is best, about 2x better than 0.05, since it moves faster without becoming unstable. 0.3 blows up. Each step jumps over the minimum and lands further away on the other side. Once each step makes the error bigger, it compounds every epoch until the weights overflow to NaN. With a simple cost like w^2, the update is $w \leftarrow w(1 - 2 \cdot lr)$, so w shrinks towards the minimum when $|1 - 2 \cdot lr| < 1$ and grows every step otherwise. The network behaves the same way along its steepest direction. Our cost is MSE without the ½ in front, so our gradients are twice as large, and lr 0.3 here behaves like 0.6 would with ½·MSE.

Summary:

Depth and learning rate had the biggest effect. The activation function had a moderate effect, mainly that logsig was slow. The starting bias hardly mattered unless it was large enough to saturate the neurons. The best single change was adding layers (3 x 20 got the test MSE down to 1.58e-4), and raising the learning rate to 0.1 was a close second with no extra compute.

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from scipy.io import loadmat
```

In [14]:
```
# Load Data
ATTRS = ["a1", "a2", "a3", "a4", "a5", "a6"]

def to_df(arr):
    return pd.DataFrame(np.asarray(arr), columns=ATTRS + ["class"]).astype(int)

def load_data(path):
    m = loadmat(path)
    keys = [k for k in m if not k.startswith("__")]
    out = {}
    for name in ["Data1", "Data2", "Data3"]:
        n = name.lower()
        train_key = next(k for k in keys if n in k.lower() and "train" in
k.lower())
        test_key  = next(k for k in keys if n in k.lower() and "test" in
k.lower())
        out[name] = (to_df(m[train_key]), to_df(m[test_key]))
    return out
```

In [5]:
```
# GINI calculations
def gini_of_labels(y):
    # computing GINI impurity
    y = np.asarray(y)
    if len(y) == 0:
      return 0
    _, counts = np.unique(y, return_counts=True)
    p = counts / len(y)
    return 1.0 - sum(p ** 2)

def gini_index_for_attribute(X_col, y):
    # TODO: compute weighted GINI for one attribute
    X_col, y = np.asarray(X_col), np.asarray(y)
    n = len(X_col)

    if n == 0:
      return 0

    unique_vals = np.unique(X_col)
    gini_index = 0.0

    for v in unique_vals:
      mask = v == X_col

      # normalising the amount of times this attribute occurs in X_col
      weight = sum(mask) / n

      # for this particular attr, we find the gini using only output vals that
      # correspond to this attr
      gini_for_attr = gini_of_labels(y[mask])
```

```
        gini_index += weight * gini_for_attr

    return gini_index

def best_attribute(X, y, attr_names):
    # return best attribute based on GINI
    best_gini = float("inf")
    best_attr = None
    for attr in attr_names:
        gini = gini_index_for_attribute(X[attr].values, y)
        if gini < best_gini:
            best_gini = gini
            best_attr = attr
    return best_attr
```

In [6]:
```
# GINI table for Data1, Data2, Data3
def compute_gini_table(df_train):
    y = df_train["class"].values
    return pd.Series(
        {a: gini_index_for_attribute(df_train[a].values, y) for a in ATTRS}
    )
```

In [7]:
```
# Build Decision Tree
def build_tree(X_train, y_train):
    # develop decision tree using training data (recursive GINI partitioning)
    attr_names = list(X_train.columns)
    y = np.asarray(y_train)
    return _grow(X_train, y, attr_names, parent_majority=None)


def _grow(X, y, attr_names, parent_majority):
    # if this is an empty node, use the parent's most common class
    if len(y) == 0:
        return {"leaf": parent_majority}

    # most common class: count each class, take the one with the highest count
    classes, counts = np.unique(y, return_counts=True)
    majority = int(classes[np.argmax(counts)])

    # if all same class, or no questions left to ask, stop
    if len(classes) == 1 or len(attr_names) == 0:
        return {"leaf": majority}

    best = best_attribute(X, y, attr_names)

    # splitting doesn't make things purer so stop
    if gini_index_for_attribute(X[best].values, y) >= gini_of_labels(y) - 1e-12:
        return {"leaf": majority}

    node = {"attr": best, "children": {}, "majority": majority}
    remaining = [a for a in attr_names if a != best]
    for v in np.unique(X[best].values):
        mask = (X[best] == v).values
```

```
        node["children"][int(v)] = _grow(X[mask], y[mask], remaining, majority)
    return node
```

In [8]:
```
# Model Testing
def classify(model, X_test):
    # traverse the decision tree to classify samples
    preds = []
    for _, row in X_test.iterrows():
        node = model
        while "leaf" not in node:
            v = int(row[node["attr"]])
            if v not in node["children"]:
                preds.append(node["majority"])
                break
            node = node["children"][v]
        else:
            preds.append(node["leaf"])
    return np.array(preds)
```

In [9]:
```
# Test Accuracy of Decision Tree
def evaluate(y_true, y_pred):
    # compute accuracy
    return float(np.mean(np.asarray(y_true) == np.asarray(y_pred)))
```

In [10]:
```
import networkx as nx

def visualize_tree(tree):
    G = nx.DiGraph()
    labels = {}        # node id -> text shown in the node
    edge_labels = {}   # (parent, child) -> attribute value on the branch
    counter = [0]      # gives every node a unique id

    def add(node):
        nid = counter[0]
        counter[0] += 1
        if "leaf" in node:
            labels[nid] = f"class {node['leaf']}"
        else:
            labels[nid] = node["attr"]
        G.add_node(nid)
        if "children" in node:
            for v, child in node["children"].items():
                cid = add(child)
                G.add_edge(nid, cid)
                edge_labels[(nid, cid)] = v
        return nid

    add(tree)

    # layered layout: root at the top, each level one row lower
    depth = nx.single_source_shortest_path_length(G, 0)
    levels = {}
    for n, d in depth.items():
        levels.setdefault(d, []).append(n)
    pos = {}
```

```
    for d, nodes in levels.items():
        for i, n in enumerate(nodes):
            pos[n] = ((i + 1) / (len(nodes) + 1), -d)

    colors = ["lightgreen" if labels[n].startswith("class") else "lightblue" for n
in G.nodes]
    plt.figure(figsize=(16, 8))
    nx.draw(G, pos, labels=labels, node_color=colors, node_size=1500, font_size=8,
arrows=False)
    nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=8)
    plt.title("Decision tree (Data1, GINI)")
    plt.show()
```

In [17]:
```
# Load Data (Data1, Data2, Data3)
data = load_data("DATASET.mat")     # {"Data1": (train_df, test_df), ...}


# compute and print the GINI table for all three datasets
table = pd.DataFrame(
    {name: compute_gini_table(train) for name, (train, _) in data.items()}
).T.round(4)
table["root"] = table[ATTRS].idxmin(axis=1)
display(table)
```

	a1	a2	a3	a4	a5	a6	root
Data1	0.1863	0.4935	0.4949	0.4620	0.4861	0.4664	a1
Data2	0.4928	0.4899	0.4885	0.4834	0.4900	0.4940	a4
Data3	0.4690	0.4639	0.4725	0.4447	0.4685	0.4691	a4

This is the table observed:

	a1	a2	a3	a4	a5	a6	root
Data1	0.1863	0.4935	0.4949	0.4620	0.4861	0.4664	a1
Data2	0.4928	0.4899	0.4885	0.4834	0.4900	0.4940	a4
Data3	0.4690	0.4639	0.4725	0.4447	0.4685	0.4691	a4

Root Attribute:

Data1: a1. It scores 0.1863, while every other attribute is above 0.46. Data2: a4. It has the lowest score (0.4834). Data3: a4. It has the lowest score (0.4447).

Is GINI a robust measure for difficult datasets?

No. GINI works well when one attribute is informative on its own, and it struggles when that is not true.

Data1: The rules are simple, so one attribute (a1) separates the classes far better than the rest. GINI finds it, and the tree reaches 100% test accuracy. Data2: All six scores sit between 0.4834 and 0.4940, which is very close to 0.5, the worst possible GINI for two classes. No attribute helps on its own, because the class depends on a global count across attributes, not on any single one. GINI scores attributes one at a time, so it cannot observe that. The choice of a4 is essentially arbitrary, since the gap to the next attribute is about 0.005. A tree built this way will generalize poorly. Data3: a4 stands out somewhat (0.4447 versus about 0.46 to 0.47 for the others), so GINI still gives a usable signal. But the scores are much closer to 0.5 than in Data1. The noisy labels blur the differences, so the choice is less reliable than on clean data.

GINI is a greedy, one-attribute-at-a-time measure. It is reliable when a single attribute carries the signal (Data1), weaker with noise (Data3), and unreliable when the pattern only shows up through combinations of attributes (Data2).

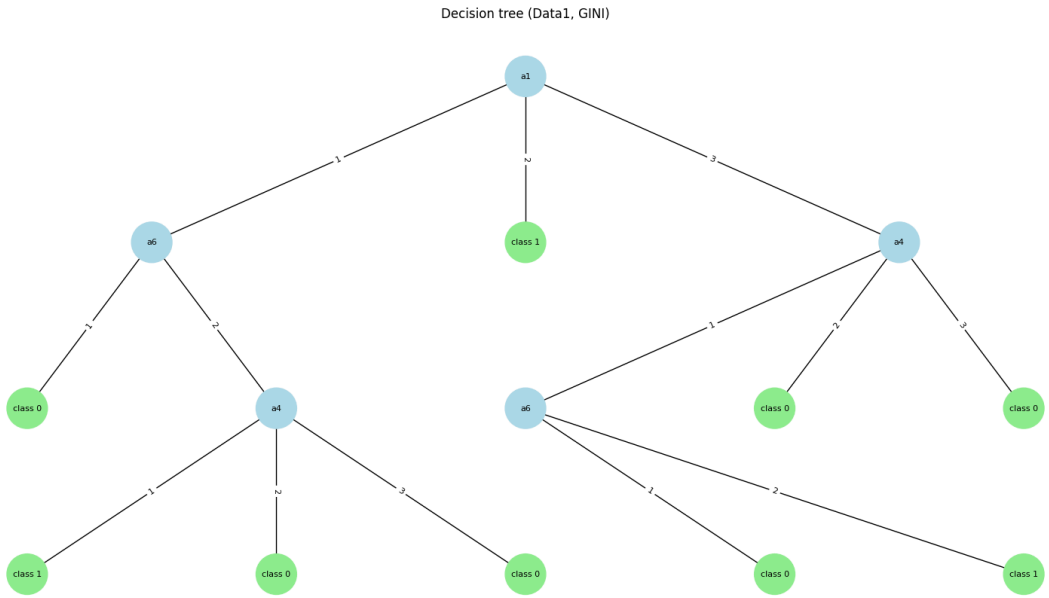
In [18]:
```
# Build Tree on Data1 training data
train1, test1 = data["Data1"]
tree = build_tree(train1[ATTRS], train1["class"].values)

# Classify Data1 test data
y_pred = classify(tree, test1[ATTRS])

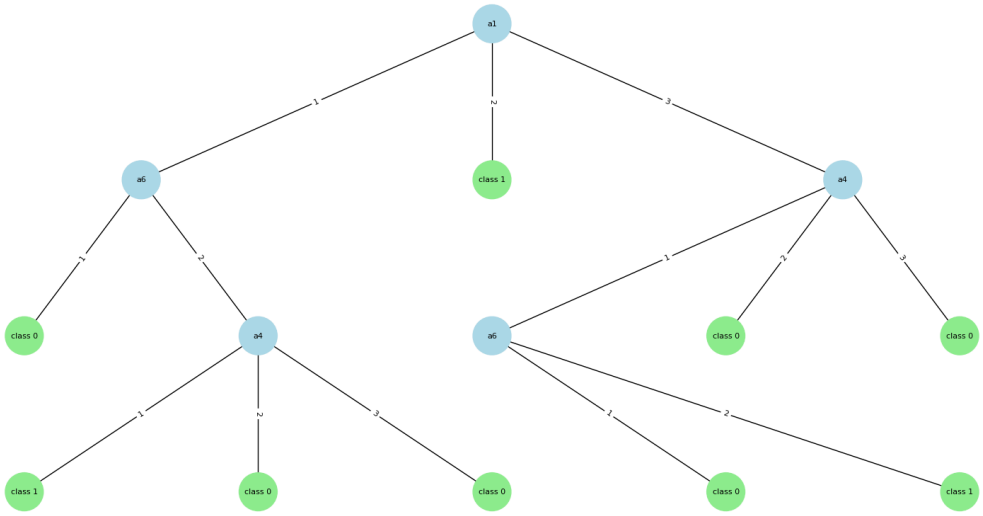
# Evaluate Model
acc = evaluate(test1["class"].values, y_pred)
print("Data1 test accuracy:", acc)

# Visualize Tree
visualize_tree(tree)
```

```
Data1 test accuracy: 1.0
```



Decision tree (Data1, GINI)



Q3: Image Classification (CIFAR-10)

Compare four scikit-learn classifiers (Decision Tree, ANN, SVM, kNN) on CIFAR-10.

CIFAR-10 has 60,000 32x32 colour images across 10 classes (50,000 train / 10,000 test).
Dataset info: https://www.cs.toronto.edu/~kriz/cifar.html

Note: SVM and kNN can be slow to train on the full 50,000-image training set with raw pixel features. Consider training on a subsample (e.g. 5,000-10,000 images) if runtime is an issue, and say so in your report.

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

from tensorflow.keras.datasets import cifar10   # only used to download the data

CLASS_NAMES = ["airplane", "automobile", "bird", "cat", "deer",
               "dog", "frog", "horse", "ship", "truck"]
```

In [3]:
```
def load_cifar10():
    # load CIFAR-10 train/test images and labels
    (X_train, y_train), (X_test, y_test) = cifar10.load_data()
    return X_train, y_train.ravel(), X_test, y_test.ravel()   # labels: (N,1) ->
(N,)

# load the data
X_train, y_train, X_test, y_test = load_cifar10()
print(X_train.shape, X_test.shape)    # (50000, 32, 32, 3) (10000, 32, 32, 3)

# subsample the training set for runtime reasons
rng = np.random.default_rng(42)
idx = rng.permutation(len(X_train))[:10000]
X_train, y_train = X_train[idx], y_train[idx]
print("Training on", len(X_train), "images")
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 1749s 10us/step
(50000, 32, 32, 3) (10000, 32, 32, 3)
Training on 10000 images
```

The dataset consists of 32 x 32 colour images from CIFAR-10 of 10 categories (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).

They must be classified using four scikit-learn classifiers: a Decision Tree, an Artificial Neural Network (ANN), a Support Vector Machine (SVM), and k-Nearest Neighbours (kNN). The same data, preprocessing, and evaluation protocol were used for all four so that the comparison is fair.

The standard split was used: 50,000 training images and 10,000 test images, with 1,000 test images per class.

Training subsample. Raw-pixel SVM and kNN scale poorly with training-set size. SVM training time grows roughly quadratically with the number of samples, and kNN must compare every test image against every stored training image. We therefore trained all four models on a random subsample of 10,000 of the 50,000 training images (a fixed random permutation with seed 42), which is the upper end of the range allowed in the assignment. Using the same subsample for every classifier keeps the comparison fair. The full 10,000-image test set was used for evaluation, so the reported accuracies are not affected by the subsampling of the test side. Because the subsample is random, the classes are only approximately balanced in it, although each class should have close to 1,000 images.

In [4]:
```
# Preprocess
def preprocess(X):
    X = X.reshape(len(X), -1)        # (N, 32, 32, 3) -> (N, 3072): one row per
image
    return X.astype("float32") / 255.0   # pixel values from 0-255 to 0-1

# preprocess train & test features
X_train_p = preprocess(X_train)
X_test_p = preprocess(X_test)

# standardize: fit on TRAIN only, then apply the same transform to test
scaler = StandardScaler()
X_train_p = scaler.fit_transform(X_train_p)
X_test_p = scaler.transform(X_test_p)
```

Feature preparation.

Flattening. scikit-learn classifiers expect one feature vector per sample, so each image (3x32x3) was flattened into a vector of 3,072 raw pixel values. This discards all spatial structure, which turns out to matter for the results below.

Scaling. Pixel values were divided by 255 to lie in [0, 1], then standardised with StandardScaler (zero mean, unit variance per feature). The scaler was fitted on the training subsample only and then applied unchanged to the test set, so no test-set information influenced training.

In [5]:
```
# Define the four classifiers to compare
classifiers = {
    "Decision Tree": DecisionTreeClassifier(max_depth=20, random_state=42),
    "ANN": MLPClassifier(hidden_layer_sizes=(256,), max_iter=50,
                         early_stopping=True, random_state=42),
    "SVM": SVC(kernel="rbf", C=10, gamma="scale"),
    "kNN": KNeighborsClassifier(n_neighbors=5, n_jobs=-1),
}
```

Decision Tree: Limiting depth stops the tree from growing until it memorises the training data (an unrestricted tree on 3,072 features would fit the training set perfectly and generalise badly).

ANN (MLP): A single hidden layer is a simple baseline. Early stopping holds out 10% of the training data as a validation set (so the network effectively trains on about 9,000 images) and stops when validation score stops improving, which limits overfitting.

SVM: The RBF kernel allows non-linear decision boundaries. C=10 allows a moderately flexible margin, and gamma='scale' sets the kernel width from the data variance.

kNN: k=5 is a common default that smooths over some noise without blurring class boundaries too much.

In [6]:
```
# Train + evaluate each classifier
results = {}

for name, clf in classifiers.items():
    print("---------------------------------")
    print(f"Learning <{name}>")
    print("---------------------------------")
    # fit clf on training data
    clf.fit(X_train_p, y_train)


    print("Classifying test data......")
    # predict on test data
    y_pred = clf.predict(X_test_p)

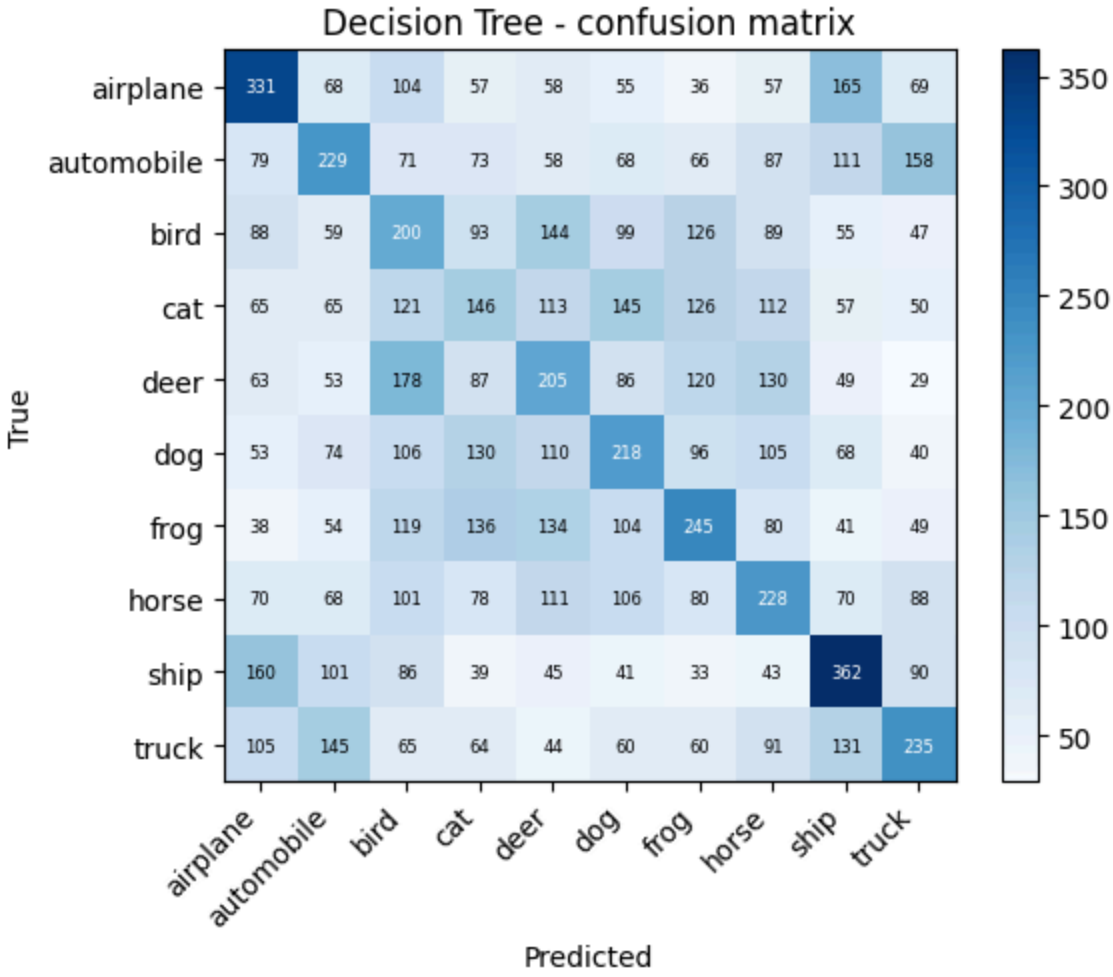

    # compute evaluation metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average="macro", zero_division=0)
    recall = recall_score(y_test, y_pred, average="macro", zero_division=0)
    f_measure = f1_score(y_test, y_pred, average="macro", zero_division=0)

    print("Evaluating results.....")
    print("Accuracy:", accuracy)
    print("Precision:", precision)
    print("Recall:", recall)
    print("F-measure:", f_measure)
    print("Confusion Matrix:")
```

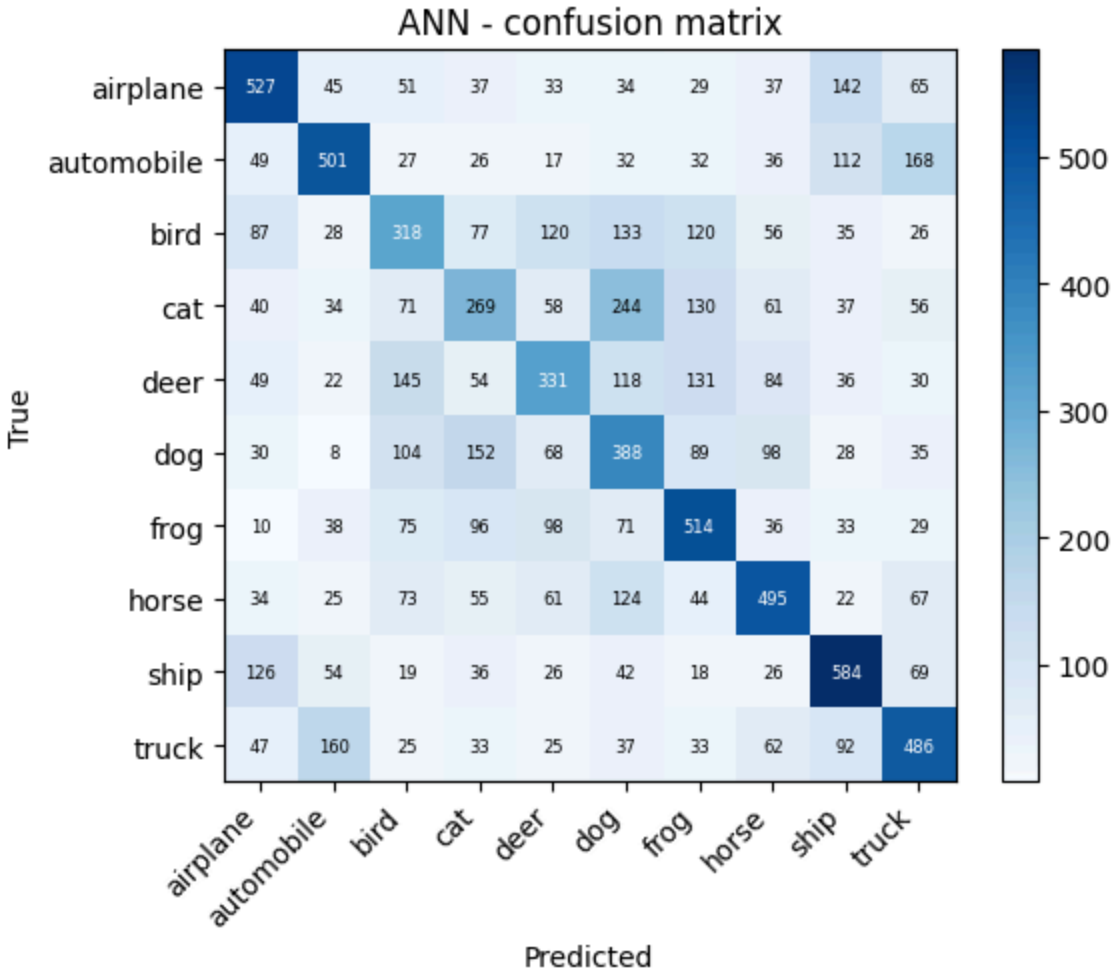
```
    # TODO: print/plot confusion matrix
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(6, 5))
    plt.imshow(cm, cmap="Blues")
    plt.colorbar()
    plt.xticks(range(10), CLASS_NAMES, rotation=45, ha="right")
    plt.yticks(range(10), CLASS_NAMES)
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.title(f"{name} - confusion matrix")
    for i in range(10):
        for j in range(10):
            plt.text(j, i, cm[i, j], ha="center", va="center", fontsize=6,
                     color="white" if cm[i, j] > cm.max() / 2 else "black")
    plt.tight_layout()
    plt.show()


    results[name] = {"accuracy": accuracy, "precision": precision, "recall":
recall, "f_measure": f_measure}
```

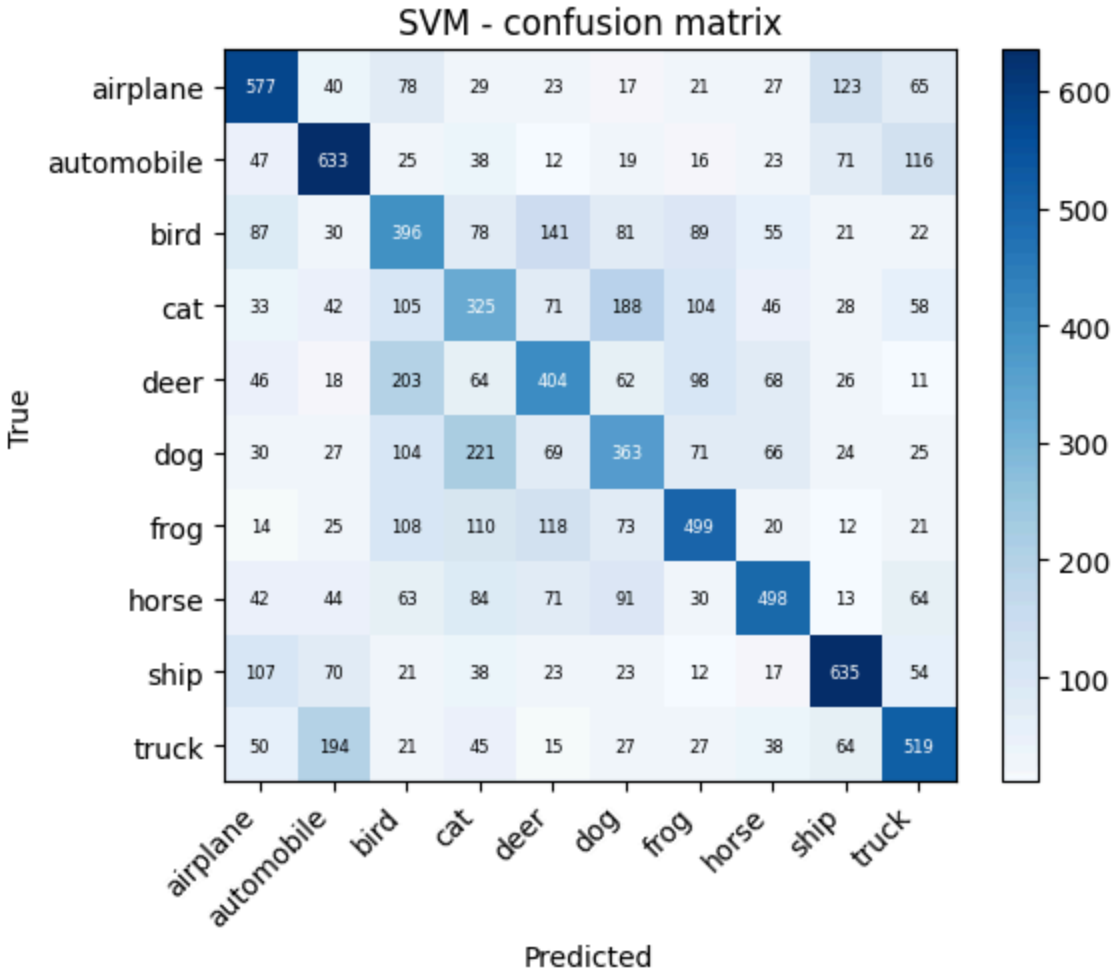
```
----------------------------------
Learning <Decision Tree>
----------------------------------
Classifying test data......
Evaluating results.....
Accuracy: 0.2399
Precision: 0.23950087027602435
Recall: 0.2399
F-measure: 0.23924611499951398
Confusion Matrix:
```



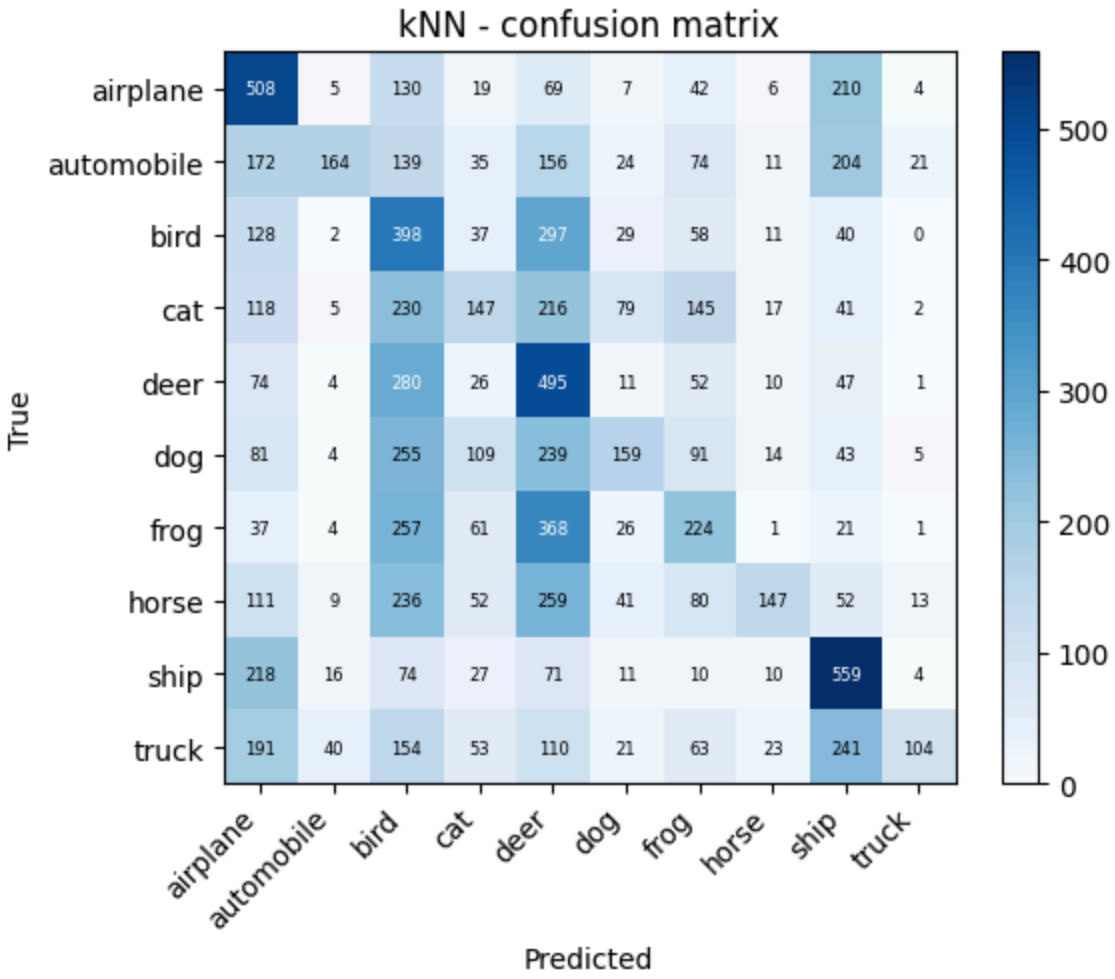

```
----------------------------------
Learning <ANN>
----------------------------------
Classifying test data......
Evaluating results.....
Accuracy: 0.4413
Precision: 0.440288017811637
Recall: 0.4413
F-measure: 0.43933475854246434
Confusion Matrix:
```




```
-----------------------------------
Learning <SVM>
-----------------------------------
Classifying test data......
Evaluating results.....
Accuracy: 0.4849
Precision: 0.4864905399584744
Recall: 0.4849
F-measure: 0.48492036553970264
Confusion Matrix:
```



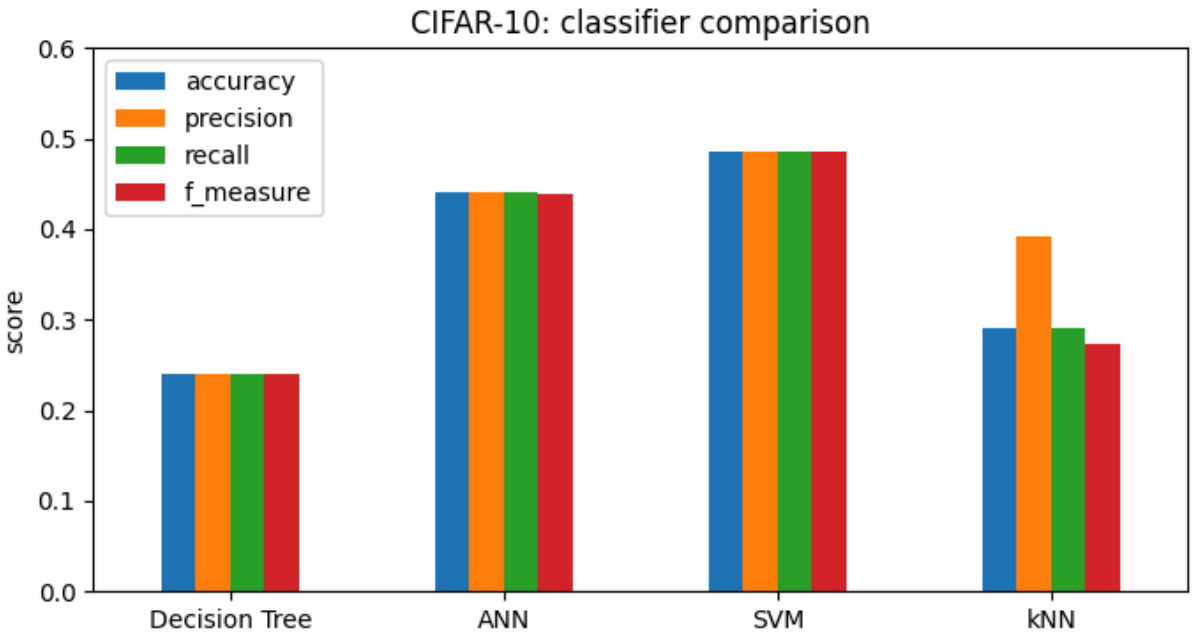

```
-----------------------------------
Learning <kNN>
-----------------------------------
Classifying test data......
Evaluating results.....
Accuracy: 0.2905
Precision: 0.39190966189481674
Recall: 0.29050000000000004
F-measure: 0.2728529559199175
Confusion Matrix:
```



In [7]:
```
# Summary table comparing all four classifiers
summary = pd.DataFrame(results).T.round(4)
display(summary)

summary.plot(kind="bar", figsize=(8, 4))
plt.title("CIFAR-10: classifier comparison")
plt.ylabel("score")
plt.xticks(rotation=0)
plt.ylim(0, 0.6)
plt.legend(loc="upper left")
plt.show()
```

	accuracy	precision	recall	f_measure
Decision Tree	0.2399	0.2395	0.2399	0.2392
ANN	0.4413	0.4403	0.4413	0.4393
SVM	0.4849	0.4865	0.4849	0.4849
kNN	0.2905	0.3919	0.2905	0.2729



Analysis:

All four models receive the same input: a flat list of 3,072 numbers. A pixel at position (10, 10) can belong to a sky in one image and to a cat's fur in another, because objects move, rotate, change size and appear against different backgrounds. No single pixel, or simple function of a few pixels, reliably identifies an object. This explains why even the best model stays below 50%. The models differ in how well they can combine pixels, and that is what separates them.

SVM (48.5%): best overall

The RBF kernel lets the SVM build smooth, non-linear boundaries that depend on all 3,072 features at once. SVMs also work well in high-dimensional spaces because the margin-based objective helps against overfitting even when features are numerous. It was the best or near-best model in almost every class, with the clearest advantages on automobile (63.3%), ship (63.5%) and airplane (57.7%). It also had the highest and most balanced precision and recall (0.4865 and 0.4849), meaning it did not strongly favour any particular class.

ANN (44.1%): second best

The hidden layer learns weighted combinations of pixels, so it can detect simple colour and brightness patterns that a tree cannot. It came within about 4 points of the SVM and was actually better on dog (38.8% vs 36.3%) and frog (51.4% vs 49.9%). It is a little behind the SVM overall, probably for these reasons: only about 9,000 images were used for fitting because 10% went to the early-stopping validation set; a single hidden layer cannot build up hierarchical features; and an MLP on flat pixels has no built-in notion of spatial locality. A larger or deeper network, more data, or longer training might narrow the gap, but we did not test this.

6.4 kNN (29.1%): unbalanced predictions

kNN classifies by Euclidean distance in 3,072-dimensional pixel space. Two images are "close" when their overall colours and brightness are similar, not when they show the same object. In very high dimensions, distances also become less discriminative. The confusion matrix shows a distinctive failure pattern. By summing the columns, kNN predicted the class "deer" about 2,280 times, "bird" about 2,150 times, "airplane" about 1,640 times and "ship" about 1,460 times, while predicting "truck" only about 155 times, "automobile" about 250 times and "horse" about 250 times (out of 10,000 predictions; a balanced predictor would give about 1,000 each). Images with plain, uniform backgrounds (blue sky or water, green or brown foliage) lie close to many other images, so they act as common nearest neighbours and attract many predictions. Objects such as cars and trucks, which appear in many colours against cluttered street backgrounds, rarely end up as nearest neighbours of one another. This is a known effect of distance-based methods in high dimensions. It also explains the per-class accuracies: kNN does well on deer (49.5%), ship (55.9%) and airplane (50.8%) but poorly on truck (10.4%), horse (14.7%) and cat (14.7%).

This also explains why kNN's macro precision (0.39) is much higher than its recall (0.29). The rarely predicted classes (automobile, truck, horse) are right most of the time when predicted (roughly 59–67% of the time), which lifts the macro-average of precision. The heavily over-predicted classes (bird, deer) have low precision (about 18% and 22%), but those low values are only two of the ten classes in the average, while recall is limited for every class that kNN under-predicts. For kNN, accuracy is therefore a more reliable single summary than precision.

6.5 Decision Tree (24.0%): weakest

A decision tree makes a sequence of threshold tests on single pixels. A single pixel carries almost no information about object identity, so the splits are weak, and many tests across the 3,072 features are needed to separate the classes. It was the lowest-scoring model, and its confusion matrix is the least concentrated on the diagonal (for example, cat 14.6% and bird 20.0%). Its best classes, ship (36.2%) and airplane (33.1%), are the ones where a global colour cue (blue background) helps. Limiting max_depth to 20 reduces memorisation but cannot fix the fundamental mismatch between axis-aligned pixel splits and image data.

Patterns in the confusion matrices (shared across models)

Several confusions appear in every matrix, which suggests they come from the data and not from any one algorithm:

Cat and dog: For the SVM, 188 cats were labelled as dogs and 221 dogs as cats. For the ANN, 244 cats were predicted as dogs. These animals have similar body shapes, fur textures and colours, and they appear in similar indoor and outdoor settings. Cat is the weakest or near-weakest class for the tree, ANN and SVM. Automobile and truck. In the SVM matrix, 194 trucks were predicted as automobiles and 116 automobiles as trucks (168 and 160 in the

ANN). Both are vehicles with wheels, similar colours and similar road backgrounds. Airplane and ship. The SVM mislabelled 123 airplanes as ships and 107 ships as airplanes. In both categories, large areas of the image are plain blue or grey (sky or sea), which pixel-level methods mostly see as the same pattern. Bird, deer and horse (animals on natural backgrounds). For example, 203 deer were predicted as birds by the SVM. These classes share green and brown backgrounds, and without shape information they are hard to separate.

In short, the models rely mainly on colour and background, not on object shape, so categories that share backgrounds or colour palettes are confused with each other, and categories with a distinctive background (ship, airplane, and for some models automobile) are classified best.